RAGstofan

Asking Statistics Iceland in plain English or Icelandic

Gagnaþon 2026 · Track 2: an agentic AI over Statistics Iceland's PX-API

What we set out to do

Statistics Iceland (Hagstofa Íslands) publishes more than 2,000 tables through its PX-API. The numbers are there, but finding them means knowing which table holds them, which of its dimensions and categories to choose, and often doing a calculation on top. RAGstofan lets anyone ask in plain English or Icelandic ("*Hve mörgum fleiri karlar en konur búa á Íslandi?*") and get back the official figure, its unit, the table it came from with a link, a chart, and every step that produced it.

Our approach: a strong harness around the model

We did not rely on a large model knowing the answer. We built a **harness**: a pipeline in which the language model makes small, checkable decisions, each with a fixed output schema, and code does everything that can be checked or computed. The model reads, plans and chooses; it never does arithmetic and never has the last word on a number. Code checks each of its choices against the table, does every calculation in Python, and compares every number in the written answer with the data before it is shown. The harness runs on open models through any OpenAI-compatible endpoint (Qwen models through Ollama on an NVIDIA DGX Spark, a desktop AI computer with the GB10 chip; the same code runs on a laptop), and every answer shows its source table, categories, calculation and steps, so a user can check it.

Our main focus was finding the correct table. Once the right table is found, reading the right number from it is a much smaller problem; with the wrong table, no model can give the right answer. Most of our work, and most of our measuring, went into the table search.

How we find the table

In short. Think of how a person would look for the right table. First they check whether they already know where it is: our *dictionary* links everyday words to the right tables. If not, they search the list of tables by its words. If that fails, they read through the whole list and pick the most likely ones. We do the same, in that order. Every time a table comes up, the model is asked a plain question before we use it: *can this table answer the question?* If the answer is no, we move on to the next way of looking. If no table can answer the whole question, it probably needs more than one, so we split it into smaller questions and look for a table for each. If we still cannot find one, we show the user the best candidates and let them choose. The rest of this section explains each piece.

The dictionary. A large share of our effort went into a bilingual dictionary (`data/keywords.json`) that maps the words people actually use to Statistics Iceland's subjects and tables. It holds 212 subjects with 2,810 English and Icelandic words and inflections (*verðbólga, verðbólgan, inflation, cost of living* → consumer price index → VIS01000), and for 171 of them the tables that answer it, checked against the catalogue's metadata. It closes the gap between everyday language and official titles, which keyword and vector search cannot close on their own.

Three ways to find a table, each double-checked. We first try to answer the whole question from a single table. To find it, we try three ways in turn, from the most reliable to the broadest, and stop at the first one that works:

1. **Look it up in the dictionary.** If the question uses a word the dictionary knows (*verðbólga*), we take the table listed for it.
2. **Search for it.** If that fails, we search all the tables by their titles and contents (two kinds of search, explained below) and take the best match.
3. **Ask the model to pick.** If that fails too, we show the model a list of all the tables and ask it for the best three.

Each time a table is suggested, the model looks at what the table contains (its title, years and categories) and answers one question: *can this table answer the question, yes or no?* Only a "yes" is used. This catches the common mistake of a table that is about the right topic but holds the wrong measure.

If none of the three ways finds a table that passes, the question probably needs more than one table ("wages compared with prices"). We then split it into two parts and look for a table for each part, then three, up to four. If a part still has no table, we show the user the best candidates and let them choose.

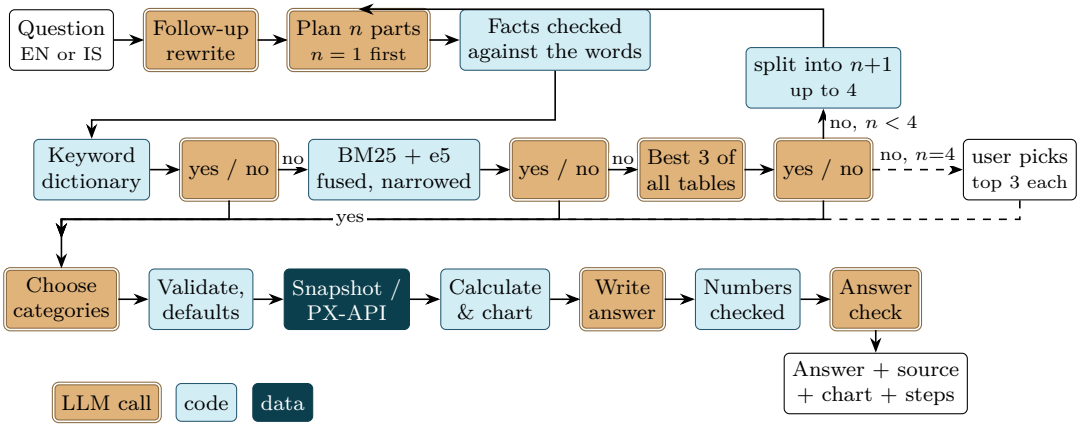


Figure 1: RAGstofan's pipeline. Each part of a question goes through the table finder (middle row) in parallel; a yes from any method, or the user's pick, leads to reading the numbers (bottom row). Python does all arithmetic.

The two searches. *BM25* is classic keyword search: it scores a table by how many of the question's words appear in its text, weighting rare words ("*gistinætur*") above common ones ("*number*"). It reads each table's titles in English and Icelandic, its dimension names and its category labels (every municipality, product or age group), so "Reykjavík" finds the tables that list it. We added prefix matching for Icelandic inflections and compounds (*Reykjavík* ∼ *Reykjavíkurborg*, *atvinnuleysið* ∼ *atvinnuleysi*), synonyms (*búa* → population), and a preference for tables that match every word and for recent tables. *e5* (`multilingual-e5-large`) is search by meaning: a neural model turns a short description of each table (titles, dimensions, period, sample categories) into a vector once, and the question into a vector each time; the tables whose vectors lie closest are the hits. It finds tables whose titles use other words than the question, and matches across languages. BM25 is better at exact terms, e5 at paraphrase, so we run both and merge their rankings: each table scores $1/(60 + rank)$ from each list, and the scores are added.

After the table. The model chooses the categories (year, place, sex, ...); code checks each choice and applies the user's own words (only the places named, the period asked, quantity or value). Code does the arithmetic (differences, changes, shares, sums, rankings, figures per inhabitant across two tables), picks the chart, and checks the written answer's numbers. Figure 1 shows the whole path.

Connection to Statistics Iceland. A crawler walked the PX-API's table tree once and stored every table's ID, titles in both languages, dimensions, periods and category labels (the catalogue that search runs over). For a chosen table the app sends a `GET` for its metadata and a `POST` asking only for the categories needed, in json-stat2 format, to `https://px.hagstofa.is/pxen/api/v1/en/<path>/<table>.px`. Calls are spaced at least one second apart, retried on `429`, and cached. By default the same documents are read from the organisers' snapshot of every table (18 October 2026), and the live API is asked only for tables or periods the snapshot lacks.

Interfaces. A website with charts and voice input, an HTTP API including the Track 2 endpoint `/spyrja_px_api`, an iPhone and iPad app, and an MCP server so Claude, ChatGPT and other agents can call it.

Test sets we built

Beyond the organisers' 60 questions, we built our own question sets to test table finding at scale, without writing answer keys by hand:

- **Questions written from the tables.** A language model read each sampled table's titles and dimensions from the catalogue and wrote three questions about it, in English and Icelandic: a plain lookup, a change between two periods, and a question about one of its categories. The table it was written from is the right answer. It also reworded each organiser question three ways (shorter, colloquial, with a typo), and wrote questions that only one of two near-identical tables can answer (base year, monthly vs yearly, old vs new series), the confusion behind most of our misses. In all about 1,300 questions over 141 tables, split by table into a half we fix against and a frozen half we only measure on.
- **Questions from Statistics Iceland's news.** We read the newest news releases on Statistics Iceland's website, and a language model wrote questions and answers from each, in English and Icelandic. A question was kept only if its answer appears word for word in the article, so the answer key is Statistics Iceland's own. These 108 questions use real topics and real phrasing, and are kept for measuring only.

Results

The organisers' benchmark. The organisers' 60 questions are over Statistics Iceland's data (release of 18 October 2026): 20 easy, 20 medium and 20 hard, each asked in English and Icelandic, with official answers. They range from single lookups (population on 1 January) to comparisons, changes, shares and figures per inhabitant across two tables. An answer counts as right only if its number matches the official one; "right table" means the source is among the accepted tables.

The final system. On a subset of 10 questions drawn at random from the organisers' 60, the system we are submitting gave **8 right answers** and 1 wrong answer, and found **the right table for 9 of the 10**. This is all the benchmarking we did on the final system: its table finder and question splitting were redesigned late in the hackathon, and there was no time for a full run.

Earlier versions. Before that redesign we measured earlier versions of the system, which shared the same harness but chose tables differently (all candidates shown to the model at once). These results are not for the system we are submitting, but they show what the harness achieved:

Earlier version, organisers' 60 questions, Qwen 3.6 35B-A3B (1 Nov 2026)	Right answer	Right table	Median time	LLM calls (median)
All questions	**59/60**	59/60	7.4 s	5
easy / medium / hard	20/20 / 20/20 / 19/20			
English / Icelandic	30/30 / 29/30			
Same questions, Qwen 3.8 27B	**60/60**			
Later run, 35B-A3B (one server error)	57/60	58/60	7.3 s	5
Team's own sets, 35B-A3B: core / topics / friends	10/10 / 20/20 / 10/10			

These earlier versions were developed against the same questions, so they are not an unseen test. Along the way, the first version (a 7B model, keyword search over titles only) answered 1 of our 10 development questions; the largest gains came from turning table-choice rules into code (drop seasonally adjusted, forecast and census tables unless asked; a named place must be an option; with no period asked, use the latest edition), which took a 14B model to 10/10. The same rules written into the prompt were followed unreliably. On 20 questions it had not seen, an early version first scored 11/20.

Search alone. Without any model, on 90 questions (our 30 plus the organisers' 60), the right table was among the top 15 search hits for 74 with BM25, 62 with e5 and 78 with the two merged; e5 found the right table for 6 questions where BM25 did not, which is why merging beats either alone.

Limits and next steps

The system is good at finding the correct table: in the final benchmark it found the right table for 9 of the 10 questions. Its weaknesses are downstream of that, in the calculations done once the table is found: dividing one figure by another (figures per inhabitant across two tables), summing several categories, and aggregating over a dimension (a total over all age groups or all countries when the table has none). Here the right table and the right categories can still give the wrong number, for example when a part is read but the division is never made, or when categories are summed that should not be.

Future work should focus on this step: deciding more reliably *which* calculation a question asks for, checking that the units of the figures being combined allow it (a percentage is not summed, a count is not divided by a rate), and building a test set aimed at calculations, as we built ours for table finding. A further step is choosing the model automatically by how hard the question is: a small, fast model for a plain lookup ("population of Reykjavík"), a larger one for questions that must be split, compared or calculated, so that simple questions are answered in seconds and hard ones get the strongest model.

Use of AI agents

We built RAGstofan with AI coding agents throughout: **Claude Code** (Anthropic) and **OpenAI Codex**. They wrote and changed code, ran tests and evaluations, and wrote documentation, including this report and the README; the team set the direction, decided the design and reviewed the work. This is documented in the repository's commit history: 94 of the 105 commits on the main branch carry a "Co-Authored-By: Claude" line, and work done with Codex is on branches named `codex/`.